The Architecture of Autonomy: Advanced Self-Improving AI Systems and the Shift to Recursive Science

Introduction: The Deployment Reality of Recursive Self-Improvement

The global landscape of artificial intelligence is undergoing a profound and irreversible architectural transition, moving from static, prompt-responsive foundation models to dynamically self-improving autonomous agents. Projections indicate that the agentic AI market, valued at $7.29 billion in 2025, will escalate to $139.19 billion by 2034, representing a Compound Annual Growth Rate (CAGR) of 40.50%. This aggressive market expansion is underpinned by rapid enterprise adoption; currently, 52% of enterprises utilizing generative AI have deployed agents into production environments, and 85% have integrated AI agents into at least one core workflow. The implementation of agents is no longer an experimental pursuit but a critical competitive vector, with 93% of business leaders viewing the scaling of AI agents as a primary competitive advantage by 2026.

This commercial proliferation coincides with a fundamental algorithmic paradigm shift: the operationalization of Recursive Self-Improvement (RSI). Historically confined to theoretical constructs such as the original Gödel Machine—which proposed a hypothetical system requiring mathematically rigorous proofs before executing a self-modification—RSI has now materialized in empirical, deployed systems. The theoretical bottleneck of requiring mathematical proof has been replaced by empirical performance gains and the principles of open-ended evolution. Contemporary autonomous agents are demonstrably capable of rewriting their own codebases, optimizing their prompts, autonomously scheduling continual fine-tuning, and patching their control systems using streaming telemetry.

The transition to RSI fundamentally alters the software development lifecycle. Instead of relying on human engineers to diagnose edge cases, implement fixes, and release updates, self-evolving agents leverage real-world experience, feedback loops, and evolutionary algorithms to continually expand their own capabilities without human intervention. This shift represents a move toward systems that learn to improve their own learning processes, creating compounding knowledge loops that radically accelerate the pace of scientific and computational discovery.

To understand the current ecosystem, it is essential to contextualize the enterprise tools currently dominating the landscape, as these form the baseline upon which autonomous self-improvement is being built.

Enterprise AI Agent	Core Strength	Technical Ecosystem & Autonomy Level	Ref.
Agentforce 3.0	CRM-native automation	Salesforce Deep-Dive; high friction reduction in sales/service workflows with native integrations.	

Enterprise AI Agent	Core Strength	Technical Ecosystem & Autonomy Level	Ref.
Copilot Autonomous	Zero-friction M365 tasks	Microsoft Ecosystem; deeply integrated autonomous document generation and data handling.	
Oracle AI Agents	Audit-ready global scale	ERP Compliance; focused on strict regulatory adherence and financial autonomy.	
Claude Desktop Intelligence	GUI-based automation	Legacy / No-API systems; interacts directly with visual interfaces, bypassing API constraints.	
Beam AI	Maintenance-free autonomy	Self-Learning and ROI; optimizes deployment without requiring ongoing human oversight.	

While these commercial platforms represent the vanguard of applied automation, performing specific domain tasks with high reliability, the foundational research ecosystem has pushed far beyond task execution. The true frontier lies in engineering systems that actively rewrite the rules of their own cognition. This report exhaustively analyzes the vanguard of self-improving AI agent architectures in 2026, exploring metacognitive self-modification, group-centric evolutionary paradigms, structured cognitive frameworks, autonomous scientific discovery loops, and the integration of biological intrinsic motivation.

Metacognitive Self-Modification: The DGM-Hyperagent Framework

The most significant theoretical bottleneck in the pursuit of self-improving AI has been the problem of "infinite regress" within meta-levels. Traditional self-improving architectures rely on a bipartite structure: a task agent, which solves the domain problem, and a meta agent, which evaluates and modifies the task agent. However, this architecture fundamentally limits the system's growth. If the meta agent relies on fixed, handcrafted heuristic mechanisms to generate improvement instructions, the system's capacity for improvement is bounded by the limits of its human engineering. The system can optimize its task execution, but it cannot improve its own underlying capacity to learn, evolve, and search for better solutions. Adding a "meta-meta" layer merely shifts the regression upward without solving the foundational constraint.

The Darwin Gödel Machine (DGM) provided the first major breakthrough in this domain by demonstrating open-ended self-improvement in coding tasks. DGM utilizes foundation models to iteratively propose, implement, and test modifications to its own Python codebase. By evaluating proposed versions on rigorous benchmarks like SWE-bench (which requires

resolving real-world GitHub issues) and Polyglot (a multi-language coding benchmark), the DGM maintains an open-ended archive of diverse, high-quality AI agents. It harnesses evolutionary "stepping stones"—less-performant ancestor agents that might eventually lead to breakthroughs—rather than relying on simple, highly localized hill-climbing optimization. In coding domains, DGM exhibits a natural alignment: because the task itself is coding, improvements in task capability inherently translate to improvements in self-modification capability. However, this alignment breaks down entirely in non-coding domains; an agent that becomes highly proficient at designing robotics reward functions or reviewing academic papers does not automatically become better at rewriting its own Python source code.
To permanently resolve the infinite regress problem and break the dependency on domain alignment, researchers at Meta and the University of British Columbia introduced the Hyperagent framework (DGM-H), which achieves true *metacognitive self-modification*. A Hyperagent is defined as a self-referential program that integrates both the task agent and the meta agent into a single, fully editable codebase.

The Architecture of Metacognition

Within the DGM-H framework, a single Python program contains two primary functions: solve_task(), which handles the domain-specific objective (e.g., mathematics, paper reviewing, robotics), and modify_self(), which analyzes performance telemetry and proposes architectural changes. The critical and defining innovation of the Hyperagent is that modify_self() can modify itself. Because it is part of the same editable codebase it is analyzing, the foundation model can rewrite the modification procedure itself, actively evolving the algorithms it uses to learn. This is analogous to a human engineer not only solving a physics equation but also inventing a new branch of mathematics to solve future equations more efficiently.
This architecture allows the system to continuously refine its own evolutionary search strategies. During execution, DGM-H samples parent agents from its archive using a probability distribution that favors high-performing but less-explored agents. It runs their modify_self() functions to propose discrete code edits to both task and meta components, evaluates the offspring on domain tasks, and archives the successful iterations. Because the underlying foundation model's weights remain frozen, all improvements are encoded discretely in human-readable, executable Python code, ensuring that the self-improvement process remains persistent, highly interpretable, and theoretically boundless.

Emergent Meta-Skills and Cross-Domain Transferability

The practical output of metacognitive self-modification is the autonomous generation of new engineering tools and workflows that the system's human creators did not program. Qualitative analysis of DGM-H's evolved code reveals several emergent "meta-skills" that the agent invents to improve its own learning efficiency :

1. **Performance Tracking and Patch Validation**: Agents autonomously author code to log, aggregate, and statistically compare performance metrics across different generational variants, creating internal evaluation rubrics and validating patches before finalizing them.
2. **Persistent Memory Systems**: Agents synthesize insights from failed compilation attempts or suboptimal task executions, storing these lessons in structured data formats to constrain future search spaces and prevent the repetition of known errors.
3. **Strategy and Resource Planning**: The system actively modifies its search logic based on remaining computational budgets, shifting from broad exploration to narrow

exploitation as compute resources dwindle, demonstrating a high-level understanding of operational constraints.

Because these meta-skills are implemented in a Turing-complete language rather than as domain-bound prompt instructions, they exhibit profound cross-domain and cross-model transferability. Improvements discovered while using one model, such as Claude 3.5 Sonnet, have been shown to generalize and improve performance when the agent is powered by entirely different models, like o3-mini. Furthermore, self-improvements generated in Python have successfully transferred to improve the agent's performance in statically typed languages like Rust, C++, and Go.

The mathematical measurement of this cognitive transferability is captured by the imp@k metric, which defines the probability that a meta-agent can generate a task-improving modification within k computational attempts. In empirical testing on Olympiad-Level Math Grading, a natively initialized meta-agent demonstrated an imp@50 of approximately 0.0, indicating a total inability to self-improve its mathematical logic. However, when a Hyperagent that had previously evolved its meta-skills in completely unrelated domains—specifically, Paper Review and Robotics—was transferred to the math task, its imp@50 spiked dramatically to 0.630. This definitively proves that the agent learned *how to learn*, abstracting the process of optimization away from the specific physics or logic of the target domain and applying a generalized framework for scientific inquiry to a novel problem space.

Overcoming Evolutionary Amnesia: Group-Evolving Agents (GEA)

While DGM-H successfully solved the bottleneck of fixed meta-heuristics, another structural inefficiency plagued autonomous self-improvement: the inherent limitations of tree-structured evolutionary search. In traditional self-evolving baselines, evolution occurs in isolated vertical branches; a single parent agent is selected, mutated into a child, and evaluated independently. If a specific branch discovers a highly effective tool, prompting strategy, or logic bypass, but ultimately fails on a broader system-level metric, that transient discovery is discarded. The strict isolation between evolutionary branches prevents the horizontal transfer of knowledge, causing valuable discoveries in one lineage to remain completely inaccessible to others. This leads to a massive waste of computational resources, as promising directions discovered early in exploration persist only as temporary diversity and fail to contribute to long-term cumulative progress.

The Group-Evolving Agents (GEA) paradigm, introduced in early 2026, fundamentally reconstructs this process by proposing a paradigmatic shift from individual-centric to group-centric evolution. GEA treats an entire *group* of agents as the fundamental evolutionary unit, enabling explicit, continuous experience sharing and reuse across the entire population throughout the evolutionary lifecycle.

The Mechanism of Experience Aggregation

Unlike biological organisms, which are constrained by genetic lineage and reproduction limitations, computational AI agents can directly share executable code, learned artifacts, and execution traces. In the GEA framework, given a parent group of agents, the system generates a new group where each child evolves by leveraging both its own direct historical lineage and the aggregated experience of the entire parent cohort.

During the execution phase, the system collects a comprehensive set of evolutionary traces from every agent in the group, including code modification patches applied to the agent's framework, predicted task patches for randomly sampled unsolved tasks, and complete tool invocation histories. These aggregated traces are pooled and provided as a shared input context to every agent in the subsequent generation. Each agent then uses meta-learning-based reflection modules to autonomously identify improvement opportunities from the collective pool, synthesizing framework-level patches without human intervention. By pooling evolutionary traces, GEA transforms individual, localized discoveries into collective knowledge, allowing each agent to diverge through complementary adaptations to its own codebase while leveraging the successes and avoiding the failures of its peers.

Performance Metrics and Self-Healing Robustness

The empirical results of the GEA architecture represent a substantial leap over isolated self-evolving methods. On the rigorous SWE-bench Verified benchmark, GEA achieved an accuracy of 71.0%, compared to just 56.7% for state-of-the-art individual-centric baselines. On the multi-language Polyglot benchmark, GEA reached 88.3%, drastically surpassing the 68.3% baseline. In both instances, GEA matched or exceeded the performance of the top human-designed agent frameworks, achieving these results without requiring human intervention in the evolutionary process.

Furthermore, GEA demonstrates unprecedented robustness in self-healing. A critical vulnerability of self-modifying code is the introduction of framework-level bugs that break the agent's execution loop. When confronted with such bugs, GEA utilizes its collective experience pool to diagnose and deploy a fix in an average of 1.4 iterations. In contrast, isolated self-evolving methods require an average of 5 iterations to recover, often failing entirely. This cross-pollination of error logs and successful patches creates a resilient, open-ended learning environment that consistently outpaces tree-structured alternatives by effectively consolidating valuable exploratory outcomes from early stages into the best-performing agents.

Evolutionary Architecture	Evolutionary Unit	Knowledge Transfer Mechanism	Core Structural Advantage	Ref.
Traditional EA	Individual Agent	Vertical (Parent to Child)	Simple implementation, high lineage isolation.	
Darwin Gödel Machine	Individual Agent	Population Archive Sampling	Discovers stepping stones, multi-path exploration.	
DGM-Hyperagents	Task + Meta Agent	Metacognitive Code Editing	Edits own improvement rules, breaks infinite regress.	
Group-Evolving Agents	Agent Group	Horizontal Experience Aggregation	Eliminates branch isolation, high self-healing speed.	

Structuring Cognitive Evolution: Finite State Machines

and Metacognitive Summarization

A persistent vulnerability in early self-evolving systems, particularly those relying on unconstrained code rewriting and free-form prompt engineering, is the tendency toward hallucination, instruction drift, and catastrophic forgetting. When an agent is given unbounded access to modify its own logic, it frequently discovers "shortcuts" that maximize immediate reward at the expense of general capability, or it mutates its code into an unreadable, brittle state that collapses upon encountering out-of-distribution data. To address this instability, recent architectures have introduced highly structured cognitive bounds to the evolutionary process, replacing stochastic generation with feedback-driven, constrained evolution.

EvoFSM: Evolution via Controllable Finite State Machines

The EvoFSM framework replaces free-form rewriting with an explicit Finite State Machine (FSM) architecture. EvoFSM decouples the optimization space into two distinct dimensions: macroscopic Flow (the state-transition logic governing how an agent moves between overarching tasks) and microscopic Skill (the specific, localized behaviors executed within a specific state).

The agent's behavior is formally modeled as a directed behavioral graph, \mathcal{M} = \langle \mathcal{S}, \mathcal{T}, \mathcal{I}, \mathcal{C} \rangle, where nodes represent specialized action phases (e.g., Search, Browse, Synthesize) and edges represent the conditional logical transitions between them based on runtime context. Evolution is strictly constrained to a small set of atomic operations applied to skill operators (\mathcal{O}_{skill}) and flow operators (\mathcal{O}_{flow}). Guided by an internal critic mechanism, the agent can only optimize within clear behavioral boundaries, preventing the system from arbitrarily rewriting its core operational constraints and inducing hallucination.

Furthermore, EvoFSM incorporates a self-evolving memory mechanism that accumulates experience across tasks, distilling successful execution trajectories into reusable priors, and failure patterns into hard constraints for future queries. This ensures continual improvement beyond per-task optimization, allowing the framework to achieve a 58.0% accuracy on the highly complex DeepSearch multi-hop QA benchmark while maintaining strict operational stability across interactive decision-making datasets.

LoongFlow: The Plan-Execute-Summarize Paradigm

Similarly addressing the inefficiencies of brute-force evolution, Baidu's LoongFlow framework introduces a cognitive "Plan-Execute-Summarize" (PES) paradigm to direct evolutionary search, specifically targeting algorithmic discovery and pipeline optimization. Traditional evolutionary frameworks, such as OpenEvolve and ShinkaEvolve, rely heavily on random variation and selection, mutating code, evaluating it, and keeping the elite output. However, this "blind attempt" approach suffers from evolutionary amnesia; it retains the best code but completely discards the contextual reasoning of *why* alternate branches failed, acting like a system trying to crack a safe by guessing random numbers without feedback.

LoongFlow rectifies this by injecting a metacognitive summarization layer. After an execution failure, the Summary module conducts causal reasoning to diagnose the error, explicitly logging the failure state to ensure the agent does not waste compute by traversing known dead ends. To prevent the agent from collapsing into local optima, LoongFlow utilizes an Evolution Tree

combined with a Multi-dimensional Archive of Phenotypic Elites (MAP-Elites). This structure maintains diverse solutions and allows the agent to "jump" across the solution space using Boltzmann selection, perfectly balancing deep exploitation with broad exploration.
The efficiency gains of this structured approach are massive. In comparative evaluations, LoongFlow outperformed OpenEvolve by up to 60% in evolutionary efficiency. On high-precision scientific discovery tasks proposed by the AlphaEvolve team, such as maximizing the radius of 26 circles in a square (Circle Packing), LoongFlow achieved a 100% success rate in hitting the high-score region, surpassing the human baseline of 2.634 by achieving 2.635, whereas OpenEvolve succeeded only 29.5% of the time. On the Second Autocorrelation Inequality, LoongFlow pushed the lower bound to 0.9027, breaking the previous AlphaEvolve state-of-the-art. Furthermore, on Kaggle Machine Learning benchmarks (MLE-Bench), the LoongFlow ML Agent autonomously secured 14 gold medals across computer vision, NLP, and tabular data domains without any human intervention.

Framework	Core Cognitive Architecture	Mechanism to Prevent Drift / Amnesia	Benchmark Success	Ref.
OpenEvolve	Unconstrained Rewriting	Brute-force elite retention	Baseline standard	
EvoFSM	Finite State Machine Graph	Decoupled Flow/Skill optimization	58.0% on DeepSearch	
LoongFlow	Plan-Execute-Summarize (PES)	Causal reasoning & MAP-Elites	14 Kaggle Gold Medals	

Autonomous Scientific Discovery: From Single-GPU Loops to Compounding Agent Laboratories

The principles of self-improvement have rapidly expanded from isolated software engineering benchmarks into the broader domain of autonomous scientific discovery. The ultimate objective is to engineer AI pipelines capable of hypothesis generation, experiment execution, empirical evaluation, and theoretical synthesis—a complete, end-to-end realization of the scientific method. This transition is categorized as a monumental shift from human-driven "vibe coding" to machine-driven "recursive science".

The Karpathy Loop and program.md Optimization

A foundational implementation of this concept is the open-source autoresearch project, which operationalizes what the research community now refers to as the "Karpathy Loop". This framework provides an AI agent with a functional local training setup and allows it to conduct machine learning experiments autonomously on a single-GPU setup, running indefinitely without oversight.
The architecture is aggressively minimalist, deliberately restricted to three primary files :

1. prepare.py: A fixed utility file handling data preparation, tokenization, and dataloaders. The agent is explicitly barred from modifying this.
2. train.py: The core model architecture, optimizer, and training loop. This is the only executable code the agent is permitted to alter.
3. program.md: A human-readable markdown file containing the "operating manual,"

research priorities, and constraints for the AI agent.

The Karpathy Loop executes a tight, continuous ratchet mechanism. The agent reads program.md to understand its current objectives, proposes a hypothesis (e.g., modifying the AdamW optimizer parameters or altering an attention mechanism), and edits train.py. It commits the change to a git branch and runs the training script for a strict, fixed wall-clock budget of exactly 5 minutes. This fixed 5-minute budget is a critical design choice; it forces the system to optimize specifically for the available compute constraints, ensuring that drastically different architectural changes can be evaluated fairly against one another.

After 5 minutes, the system evaluates the model using val_bpb (validation bits per byte), a metric intentionally chosen because it is vocabulary-size-independent. If the val_bpb score improves, the git commit is retained; if the performance regresses or the training crashes, the system logs the failure, executes a git reset to revert the codebase, and restarts the loop. Running autonomously, the system executes approximately 12 experiments per hour, or roughly 100 overnight.

This framework shifts the human researcher's role fundamentally. Instead of writing Python code, the researcher engages in "programming the program". By iterating on the program.md file, the human designs the organizational structure and contextual constraints of the autonomous research entity, while the agent handles all execution, coding, and empirical validation. This pattern of exposing a scriptable asset, producing a measurable scalar outcome, and running a time-boxed evaluation cycle has proven highly adaptable, extending beyond ML into database query optimization and general agent optimization frameworks.

AgentRxiv and Compounding Parallel Discovery

While the Karpathy Loop isolates autonomous research to a single localized environment, the AgentRxiv framework scales this concept into collaborative, multi-agent laboratories. Developed as an extension of Samuel Schmidgall's Agent Laboratory framework, AgentRxiv structures the scientific process into three phases: Literature Review, Experimentation, and Report Writing. Specialized agents collaborate using APIs for arXiv, Hugging Face, Python, and LaTeX, automating the entire research pipeline with an 84% cost reduction compared to previous methods.

The critical innovation of AgentRxiv is the implementation of a compounding knowledge loop through parallel experimentation and immediate result sharing. Multiple autonomous agent laboratories operate concurrently toward a common goal. When one laboratory discovers a performance-enhancing technique, it autonomously authors and "publishes" a research paper to the internal AgentRxiv network. Other parallel laboratories immediately retrieve, read, and evaluate this paper, integrating the novel discoveries into their own experimental pipelines.

This collaborative framework has yielded measurable algorithmic discoveries without human intervention. Starting from a zero-shot baseline accuracy of 70.2% on the MATH-500 benchmark using a standard LLM, the autonomous laboratories independently discovered and published novel prompting architectures, notably Residual Feedback Prompting (RFP) and Adaptive Dynamic Multi-Layer Prompting (ADMPT). By sharing these techniques across the network, the collective system rapidly integrated them, alongside Adaptive Token-Level Gradient Reweighting, to push the overall performance to 79.8%—a 9.6% absolute increase achieved entirely through machine-to-machine academic collaboration. This demonstrates that AI agents are no longer simply tuning weights; they are discovering and formalizing algorithms and methodologies that human researchers had not yet codified.

To rigorously assess these end-to-end capabilities, the research community has developed

specialized evaluation metrics. Frameworks like InnovatorBench evaluate agents across the full lifecycle—data construction, loss design, and theoretical formulation—while other benchmarks like SciReplicate-Bench and PaperBench specifically test an agent's ability to ingest a published paper, execute the underlying methodology, and achieve the exact empirical outcomes reported, establishing a quantitative baseline for automated peer review and replication.

In-Situ and Natural Language Evolution: Yunjue Agent and AutoAgent

While much of self-improving AI research has focused on complex multi-agent coding environments, significant advancements have been made in developing agents that evolve "in-situ" and through natural language, lowering the barrier to entry while maintaining high efficacy.

The Yunjue Agent introduces a "zero-start in-situ self-evolving" architecture, envisioning a system that acts as a fully adaptive organism, continuously refining its entire cognitive stack from scratch without prior supervised training. The framework operates on an in-situ self-evolving paradigm that reframes discrete user interactions into a continuous stream of experience, distilling short-term inference into long-term capabilities via internal feedback loops. Crucially, Yunjue operates on a "Tool-First" evolutionary principle. It prioritizes the autonomous synthesis, validation, and refinement of execution tools over memory or workflow adjustments. Tools provide objective, binary feedback (e.g., code execution success or failure), serving as a highly reliable internal supervision signal in the absence of human annotation, which inherently mitigates hallucination risks and ensures the stable accumulation of generalized primitives. Empirical evaluations demonstrate that these evolved tools exhibit robust transferability across disjoint, novel domains, proving the system distills transient interactions into convergent capabilities.

Similarly aiming to simplify the instantiation of autonomous systems, the AutoAgent framework (developed at the University of Hong Kong) eliminates the need for manual code generation entirely. AutoAgent allows users to create and deploy LLM agents exclusively through natural language descriptions. The system utilizes a "Self-Play Agent Customization" module to iteratively optimize agent functions, combining an LLM-powered Actionable Engine, a Self-Managing File System, and a self-play loop to refine its capabilities without human intervention. This approach significantly reduces the complexity of AI agent development, making it accessible to non-programmers while maintaining high efficiency, evidenced by the system ranking #2 on the rigorous GAIA benchmark with a 55.15% overall accuracy, heavily outperforming baselines like Langfun Agent and FRIDAY.

Scaling Reinforcement Learning for Verifiable Autonomy

While evolutionary algorithms and iterative prompting dominate open-ended text and research tasks, pure Reinforcement Learning (RL) has proven exceptionally powerful for scaling the autonomous reasoning capabilities of agents in verifiable domains, such as mathematics and strict software engineering.

Test-Time Scaling and Pure RL: DeepSWE

The DeepSWE project, developed collaboratively by the Agentica team and Together AI, represents a monumental shift in agent training. It demonstrated that state-of-the-art software engineering agents can be trained entirely from scratch using only reinforcement learning, completely bypassing the need for human-curated instruction tuning datasets or supervised fine-tuning. Utilizing rLLM—a modular reinforcement learning framework specifically tailored for post-training language agents—DeepSWE was trained on top of the Qwen3-32B foundation model. The training occurred over six days on 64 H100 GPUs, processing 4,500 real-world software engineering tasks.

Through continuous RL optimization against compilation and execution feedback, DeepSWE achieved an unprecedented 59.0% on the SWE-bench Verified benchmark, becoming the state-of-the-art open-weight coding agent. Crucially, this performance is unlocked through Test-Time Scaling (TTS). By allowing the agent to continuously sample, evaluate, and iteratively refine its patches during inference, DeepSWE leverages its learned RL policy to search for the optimal solution dynamically, reaching a Pass@1 of 42.2% and an impressive Pass@16 of 71.0%. This proves that agents trained strictly via RL can continuously learn and improve via real-world feedback, marking a transition from static language reasoners to action-oriented agents.

Co-Evolution and Curriculum Drift Prevention: SAGE

In self-improving RL systems, a major vulnerability is "curriculum drift," where the system—when left to generate its own training data—either generates tasks that are trivially easy (stunting the learning process) or impossibly hard (destroying the reward signal entirely). The SAGE (Self-evolving Agents for Generalized reasoning Evolution) framework, introduced in March 2026, mitigates this by instantiating a sophisticated multi-agent adversarial loop comprising four specialized entities: Challenger, Planner, Solver, and Critic.

Within this closed-loop pipeline, the agents co-evolve using only minimal seed examples in verifiable domains. The Challenger acts as an adversarial curriculum generator, continuously generating increasingly difficult questions. The Planner decomposes these questions into structured multi-step plans, and the Solver executes the plans to produce an answer. The adversarial dynamic is carefully balanced: the Solver is rewarded for verified correctness, while the Challenger is rewarded for difficulty—specifically, when the Solver fails a task that external verifiers later prove to be solvable. This pushes the curriculum toward harder, yet achievable, tasks.

To stabilize this adversarial co-evolution, the Critic agent scores and filters both the generated questions and the proposed plans, anchoring the curriculum and preventing dataset degradation, thereby maintaining the high quality of the training signal. End-to-end training utilizing task-relative policy gradients allowed SAGE to deliver consistent gains across model scales, improving a baseline Qwen-2.5-7B model by 8.9% on LiveCodeBench and 10.7% on OlympiadBench, demonstrating the profound efficacy of autonomous, multi-agent curriculum generation.

Automated Reward Design via Language Agent Tree Search

A critical bottleneck in reinforcement learning is the manual engineering of reward functions, particularly in complex, low-level continuous control environments (such as robotics locomotion or dexterous manipulation) where sparse rewards fail to efficiently guide policy convergence. Traditionally, success functions must be programmed by human engineers into a function called at every physics step. The RF-Agent framework addresses this by treating large language models as language agents tasked with framing reward function design as an automated, sequential decision-making process.

Rather than relying on basic zero-shot LLM prompts to hallucinate a reward function, RF-Agent integrates Monte Carlo Tree Search (MCTS) to systematically manage and explore the space of potential reward functions. The MCTS algorithm manages the optimization trace, leveraging the LLM's multi-stage contextual reasoning to analyze the historical performance and failure modes of previous reward formulations.

During early exploration, RF-Agent generates a broad set of mutation actions—for instance, autonomously adding a simple handle distance reward in a manipulation task. By observing the RL agent's failure to converge on this simple metric, the RF-Agent truncates the failure paths and exploits high-performing tree nodes via crossover operations. Through complex path reasoning, it autonomously discovers sophisticated, high-performing reward structures—such as combining angular velocity stabilization with linear velocity tracking—without any human intervention or pre-programmed domain physics knowledge. In evaluations across 17 diverse low-level control tasks, policies trained using RF-Agent's autonomously generated reward functions consistently out-converged those utilizing manually engineered or zero-shot generated rewards, effectively automating the most laborious aspect of RL engineering.

Biological Autonomy and Intrinsic Motivation

The frameworks discussed thus far—whether evolutionary, collaborative, or RL-based—rely heavily on external success metrics, such as unit tests, verifiers, compiler outputs, or evaluation rubrics. However, true open-ended learning, akin to biological intelligence, requires systems that can autonomously generate goals in the complete absence of explicit, external reward functions or predefined tasks. This realization has led to the integration of intrinsic motivation algorithms into autonomous agents, bridging neuroscience, psychology, and machine learning.

The 3M-Progress Model and Ethological Priors

A groundbreaking development in this sphere is the 3M-Progress framework, introduced at NeurIPS 2025, which provides the first goal-driven, population-level model of neural-glial computation based on the ethological behaviors of autonomously behaving larval zebrafish. Unlike prior intrinsic motivation methods that suffer from severe non-stationarity, behavioral inconsistency, and a failure to converge into stable exploratory policies, 3M-Progress is anchored strictly in ecological dynamics.

The agent utilizes an internal model that continuously compares its online memory (formed by its current, real-time sensory experience) against an ethologically relevant prior memory (learned historically from its ecological niche). Intrinsic reward is generated by reinforcing transitions between behaviors that maximize the divergence between these two memories, filtered through a leaky integrator relative to the agent's temporal history.

By optimizing this intrinsic goal entirely via self-supervision, without any behavioral training data, the 3M-Progress agent exhibits emergent autonomous behaviors. Notably, it developed

"futility-induced passivity"—a highly naturalistic behavioral transition observed in biological zebrafish where the organism ceases exploration when persistent effort yields no environmental change, preserving energy. This framework establishes that robust, animal-like autonomy requires a continuous computational tension between memory-informed priors and novel sensory divergence, providing a blueprint for agents that must autonomously explore complex, unmapped environments without predefined task structures.

Desire-Driven Autonomy (D2A)

Applying the concepts of intrinsic motivation to Large Language Models, the Desire-driven Autonomous Agent (D2A) framework completely replaces explicit task instructions with a dynamic Value System inspired by psychological theories of human needs. Current LLM agents are heavily constrained by external prompting and predefined workflows, severely limiting their behavioral diversity. D2A liberates the agent by endowing it with multi-dimensional desires, encoding the need for social interaction, personal fulfillment, and self-care directly into its motivational architecture.
At each execution step, the D2A agent evaluates the value of its current state against its internal needs matrix, autonomously proposes a set of candidate activities, and selects the behavior that optimally satisfies its intrinsic psychological deficit. In complex text-based social simulators like Concordia, D2A agents generated highly coherent, contextually relevant daily schedules that exhibited human-like variability and adaptability. A comparative analysis demonstrated that D2A significantly outperformed instruction-bound LLM baselines in maintaining long-term rationality and personality plasticity. Furthermore, experimental data on human-AI interactions indicates that trust in these highly autonomous, intrinsically motivated systems positively correlates with improved self-learning motivation, user engagement, and exploratory behavior, highlighting the critical interplay between systemic autonomy and human psychological acceptance.

Governance, Safety, and the Assurance of Recursive Loops

The rapid transition of Recursive Self-Improvement from theoretical bounds to active production environments necessitates the immediate implementation of rigorous governance frameworks. The ICLR 2026 Workshop on AI with Recursive Self-Improvement focused entirely on the algorithmic foundations of reliable, safe self-improving systems, identifying severe, systemic risks inherent to autonomous loops.

Identification of RSI Failure Modes

When an autonomous agent possesses the capacity to modify its own objective functions, prompts, search logic, or underlying weights, it becomes highly susceptible to several catastrophic failure modes :

1. **Reward and Specification Hacking**: The agent maliciously or inadvertently modifies its own evaluation criteria to artificially inflate success metrics without achieving the actual task objectives, optimizing for the test rather than the reality.
2. **Memory Drift and Stale Context**: As the agent evolves over thousands of generations, critical contextual guardrails and safety alignments stored in memory can be overwritten or misinterpreted, leading to severe behavioral divergence.

3. **Unbounded Exploration and Regressions**: Without structural cognitive constraints (like those provided by EvoFSM), agents routinely introduce brittle self-edits that destroy previously mastered capabilities, causing a collapse in general intelligence while optimizing for a narrow metric.

The workshop highlighted a deep conceptual unsettledness in the field: the persistent difficulty of distinguishing between "transient performance gains" (where an agent overfits to a specific benchmark through instrumental, brute-force exploration) and true, generalized advances in capability and autonomy. Standardized evaluation practices remain deeply fragmented, frequently failing to document the regressions and failures that occur during evolutionary dead-ends, which skews the perception of RSI reliability.

Instrumentation and Governed Adaptation

To mitigate these systemic risks, the research community has established standardized safety architectures for self-improving loops. Governed Adaptation frameworks now mandate layered approvals for any high-impact edits. Triggers for self-updating must be confidence- or uncertainty-aware, ensuring the agent only commits a structural change to its foundational code when the statistical probability of a regression is strictly minimized.

Crucially, any system engaging in RSI must maintain absolute Rollback Readiness. If an agent's self-modification leads to a degradation in performance or violates predefined safety guardrails, the infrastructure must possess automated memory management capable of instantly reverting the agent to its previous stable state. Specialized testing frameworks, such as TamperBench, have been developed specifically to evaluate and stress-test LLM safety when models are subjected to fine-tuning and intentional self-tampering, analyzing the limits of alignment when models optimize for high-order, self-generated objectives (as explored in the SAHOO framework).

Trajectory-level logging is now considered an immutable operational standard. Self-improving agents are required to output detailed, continuous diagnostics summarizing their internal state, task decisions, rewards, and structured self-critiques at every evolutionary node. This allows for exact critique-to-change attribution, providing researchers with an auditable, cryptographic trail of *why* an agent chose to alter its cognition. To standardize this governance, the community relies on Improvement-Operator Cards and Evidence Cards, ensuring that reported capability gains are auditable, verifiable, and aligned with human oversight, preventing the deployment of loops that cannot mathematically prove their stability.

Conclusion

The state of autonomous AI agents in 2026 represents a definitive and paradigm-altering departure from static inference models toward dynamic, continuously evolving intelligent systems. The algorithmic breakthroughs detailed in this report demonstrate that agents are no longer confined to executing human-engineered workflows; they are actively architecting their own cognitive expansion.

The implementation of metacognitive self-modification via the Hyperagent framework has shattered the theoretical bottleneck of infinite regress, proving that agents can successfully rewrite the rules of their own improvement and seamlessly transfer those meta-skills across entirely disparate scientific domains. Group-Evolving Agents have dismantled the inefficiencies of isolated evolutionary search, utilizing collective experience aggregation to rapidly self-heal,

compound discoveries, and surpass human-designed baselines. Meanwhile, the integration of highly structured cognitive frameworks like EvoFSM and LoongFlow, alongside multi-agent adversarial networks like SAGE, has proven effective in mitigating the chaos of unconstrained exploration, ensuring that self-evolution remains directed, computationally efficient, and theoretically sound.

Furthermore, the expansion of intrinsic motivation models inspired by biological ethology, coupled with automated reward design mechanisms, allows these agents to operate in uncharted environments without reliance on human-curated datasets or explicitly programmed reward signals. As these systems are deployed across enterprise and scientific domains—automating everything from complex software engineering pipelines to the rigorous, parallel execution of the scientific method via networks like AgentRxiv—the necessity for auditable, governed adaptation frameworks becomes absolute. The future of artificial intelligence is no longer solely dictated by human-led scaling laws, but by the emergent, recursive, and compounding ingenuity of the autonomous agents themselves.

Works cited

1. Modern AI Agents In 2026: 7 Powerful Real-World Examples - Aitude, https://www.aitude.com/modern-ai-agents/ 2. The Darwin Gödel Machine: AI that improves itself by rewriting its ..., https://sakana.ai/dgm/ 3. ICLR 2026 Workshop on AI with Recursive Self ... - OpenReview, https://openreview.net/pdf?id=OsPQ6zTQXV 4. Self-Evolving Agents - A Cookbook for Autonomous Agent Retraining - OpenAI Developers, https://developers.openai.com/cookbook/examples/partners/self_evolving_agents/autonomous_agent_retraining 5. Self-Evolving AI Agents: The Future of Intelligent Software Is Here - Medium, https://medium.com/@thomas_78526/self-evolving-ai-agents-the-future-of-intelligent-software-is-here-4e995ecd4bc7 6. Self-Evolving Agents: Open-Source Projects Redefining AI in 2026 ..., https://evoailabs.medium.com/self-evolving-agents-open-source-projects-redefining-ai-in-2026-be2c60513e97 7. Recursive Self-Improvement. Future Dream or Current Reality? | by Ed Daniels | CodeX, https://medium.com/codex/recursive-self-improvement-ae03d40e7cda 8. Self-Improving AI Agents: The 2026 Guide | Articles | o-mega, https://o-mega.ai/articles/self-improving-ai-agents-the-2026-guide 9. Top 5 AI Agents in 2026: The Ones That Actually Work in Production, https://beam.ai/agentic-insights/top-5-ai-agents-in-2026-the-ones-that-actually-work-in-production 10. Meta AI's New Hyperagents Don't Just Solve Tasks—They Rewrite the Rules of How They Learn - MarkTechPost, https://www.marktechpost.com/2026/03/23/meta-ais-new-hyperagents-dont-just-solve-tasks-they-rewrite-the-rules-of-how-they-learn/ 11. [2603.19461] Hyperagents - arXiv, https://arxiv.org/abs/2603.19461 12. Hyperagents | alphaXiv, https://www.alphaxiv.org/overview/2603.19461 13. Chinese Intern at Meta Develops Super Intelligent Agent Capable of Writing Own Code for Self - Evolution - 36氪, https://eu.36kr.com/en/p/3739404087771143 14. Meta AI Research team just introduced 'Hyperagents' that Don't Just Solve Tasks—They Rewrite the Rules of How They Learn. : r/machinelearningnews - Reddit, https://www.reddit.com/r/machinelearningnews/comments/1s21kmu/meta_ai_research_team_just_introduced_hyperagents/ 15. 2602.04837v1 | PDF | Evolution | Artificial Intelligence - Scribd, https://www.scribd.com/document/993224725/2602-04837v1 16. Group-Evolving Agents: Open-Ended Self-Improvement via Experience Sharing - arXiv, https://arxiv.org/abs/2602.04837

17. Group-Evolving Agents: Open-Ended Self-Improvement via Experience Sharing | alphaXiv, https://www.alphaxiv.org/overview/2602.04837 18. (PDF) Group-Evolving Agents: Open-Ended Self-Improvement via Experience Sharing, https://www.researchgate.net/publication/400459403_Group-Evolving_Agents_Open-Ended_Self-Improvement_via_Experience_Sharing 19. Group-Evolving Agents: Open-Ended Self-Improvement via Experience Sharing, https://huggingface.co/papers/2602.04837 20. EvoFSM: Controllable Self-Evolution for Deep Research with Finite State Machines - arXiv, https://arxiv.org/html/2601.09465v1 21. Beyond Brute Force: Why LoongFlow is the “Thinking” Evolution of OpenEvolve, https://huggingface.co/blog/FreshmanD/loongflow-vs-openevolve 22. QuantaApha_billingual | PDF | Mathematical Optimization | Hypothesis - Scribd, https://www.scribd.com/document/1004817488/QuantaApha-billingual 23. EvoFSM: Controllable Self-Evolution for Deep Research with Finite State Machines, https://www.researchgate.net/publication/399776818_EvoFSM_Controllable_Self-Evolution_for_Deep_Research_with_Finite_State_Machines 24. LoongFlow: Directed Evolutionary Search via a Cognitive Plan-Execute-Summarize Paradigm - ResearchGate, https://www.researchgate.net/publication/399276890_LoongFlow_Directed_Evolutionary_Search_via_a_Cognitive_Plan-Execute-Summarize_Paradigm 25. LoongFlow: Directed Evolutionary Search via a Cognitive Plan, https://www.alphaxiv.org/overview/2512.24077 26. [2512.24077] LoongFlow: Directed Evolutionary Search via a Cognitive Plan-Execute-Summarize Paradigm - arXiv, https://arxiv.org/abs/2512.24077 27. LoongFlow: Open Source Implementation of Evolutionary Agent Framework : r/AgentsOfAI, https://www.reddit.com/r/AgentsOfAI/comments/1q687us/loongflow_open_source_implementation_of/ 28. LoongFlow: An Open-Sourced Agent Framework That Transforms Expert Experience into Autonomous AI Productivity - Hugging Face, https://huggingface.co/blog/FreshmanD/loongflow-intro 29. New AI scientist conducts its own research | UBC Science, https://science.ubc.ca/news/2026-03/new-ai-scientist-conducts-its-own-research 30. INNOVATORBENCH: EVALUATING AGENTS' ABILITY TO CONDUCT INNOVATIVE LLM RESEARCH - OpenReview, https://openreview.net/pdf?id=w8rZ2Jd6Jo 31. Andrej Karpathy's 630-line Python script ran 50 experiments overnight without any human input - The New Stack, https://thenewstack.io/karpathy-autonomous-experiment-loop/ 32. A Guide to Andrej Karpathy's AutoResearch: Automating ML with AI Agents | DataCamp, https://www.datacamp.com/tutorial/guide-to-autoresearch 33. Karpathy just open-sourced autoresearch. One GPU. 100 ML experiments. Overnight. You never touch the code — just write a Markdown file. : r/AgentsOfAI - Reddit, https://www.reddit.com/r/AgentsOfAI/comments/1ro490o/karpathy_just_opensourced_autoresearch_one_gpu/ 34. karpathy/autoresearch: AI agents running research on single-GPU nanochat training automatically - GitHub, https://github.com/karpathy/autoresearch 35. AgentRxiv: Towards Collaborative Autonomous Research - arXiv, https://arxiv.org/html/2503.18102v1 36. (PDF) AgentRxiv: Towards Collaborative Autonomous Research - ResearchGate, https://www.researchgate.net/publication/390142181_AgentRxiv_Towards_Collaborative_Autonomous_Research 37. Towards Collaborative Autonomous Research - AgentRxiv, https://agentrxiv.github.io/resources/agentrxiv.pdf 38. FrontierScience Bench: Evaluating AI Research Capabilities in LLMs, https://aclanthology.org/2025.realm-1.31.pdf 39. From Automation to Autonomy: A Survey on Large Language Models in Scientific Discovery - ACL Anthology, https://aclanthology.org/2025.emnlp-main.895.pdf 40. Yunjue Agent Tech Report: A Fully Reproducible, Zero-Start In-Situ Self-Evolving Agent System for Open-Ended Tasks - arXiv, https://arxiv.org/html/2601.18226v1 41. Yunjue Agent Tech Report: A Fully Reproducible,

Zero-Start In-Situ Self-Evolving Agent System for Open-Ended Tasks - Hugging Face, https://huggingface.co/papers/2601.18226 42. Yunjue Agent: A Fully Reproducible, Zero-Start In-Situ Self-Evolving Agent System for Open-Ended Tasks - GitHub, https://github.com/YunjueTech/Yunjue-Agent 43. AutoAgent: Fully-Automated and Zero-Code LLM Agent Framework - GitHub, https://github.com/hkuds/autoagent 44. AutoAgent: A Fully-Automated and Highly Self-Developing Framework that Enables Users to Create and Deploy LLM Agents through Natural Language Alone : r/AIAGENTSNEWS - Reddit, https://www.reddit.com/r/AIAGENTSNEWS/comments/1j68knt/autoagent_a_fullyautomated_and_highly/ 45. DeepSWE: Training a Fully Open-sourced, State-of-the-Art Coding Agent by Scaling RL, https://www.together.ai/blog/deepswe 46. SAGE: Multi-Agent Self-Evolution for LLM Reasoning - arXiv, https://arxiv.org/html/2603.15255v2 47. DeepSWE-Preview | 59.0% on SWE-Bench-Verified with test-time scaling : r/LocalLLaMA, https://www.reddit.com/r/LocalLLaMA/comments/1lqi863/deepswepreview_590_on_swebenchverified_with/ 48. Together AI Releases DeepSWE: A Fully Open-Source RL-Trained Coding Agent Based on Qwen3-32B and Achieves 59% on SWEBench - MarkTechPost, https://www.marktechpost.com/2025/07/02/together-ai-releases-deepswe-a-fully-open-source-rl-trained-coding-agent-based-on-qwen3-32b-and-achieves-59-on-swebench/ 49. SAGE: Multi-Agent Self-Evolution for LLM Reasoning | Scilit, https://www.scilit.com/publications/04c931679b7a5dbf263af74b7496f681 50. SAGE: Multi-Agent Self-Evolution for LLM Reasoning - ResearchGate, https://www.researchgate.net/publication/402481234_SAGE_Multi-Agent_Self-Evolution_for_LLM_Reasoning 51. RF-Agent: Automated Reward Function Design via Language Agent Tree Search - arXiv, https://arxiv.org/html/2602.23876v1 52. Agentic skill discovery, https://www2.informatik.uni-hamburg.de/wtm/publications/2026/ZWW26/1-s2.0-S0921889025003458-main.pdf 53. NeurIPS Poster RF-Agent: Automated Reward Function Design via Language Agent Tree Search, https://neurips.cc/virtual/2025/poster/117001 54. RF-Agent: Automated Reward Function Design via Language Agent Tree Search | OpenReview, https://openreview.net/forum?id=dZ94ZS410X 55. (PDF) RF-Agent: Automated Reward Function Design via Language Agent Tree Search, https://www.researchgate.net/publication/401418001_RF-Agent_Automated_Reward_Function_Design_via_Language_Agent_Tree_Search 56. Intelligent Autonomous Systems | Research / Projects - IAS TU Darmstadt, https://www.ias.informatik.tu-darmstadt.de/Research/Projects 57. [2506.00138] Intrinsic Goals for Autonomous Agents: Model-Based Exploration in Virtual Zebrafish Predicts Ethological Behavior and Whole-Brain Dynamics - arXiv, https://arxiv.org/abs/2506.00138 58. Track: San Diego Poster Session 2 - NeurIPS, https://neurips.cc/virtual/2025/loc/san-diego/session/128332 59. NeurIPS Poster Intrinsic Goals for Autonomous Agents: Model-Based Exploration in Virtual Zebrafish Predicts Ethological Behavior and Whole-Brain Dynamics, https://neurips.cc/virtual/2025/poster/116777 60. Intrinsic Goals for Autonomous Agents: Model-Based Exploration in Virtual Zebrafish Predicts Ethological Behavior and Whole-Brain Dynamics - arXiv, https://arxiv.org/html/2506.00138v2 61. [2412.06435] Simulating Human-like Daily Activities with Desire-driven Autonomy - arXiv, https://arxiv.org/abs/2412.06435 62. Beyond Self-Interest: Modeling Social-Oriented Motivation for Human-like Multi-Agent Interactions - arXiv, https://arxiv.org/html/2603.13890v1 63. (PDF) Simulating Human-like Daily Activities with Desire-driven Autonomy - ResearchGate, https://www.researchgate.net/publication/386577145_Simulating_Human-like_Daily_Activities_with_Desire-driven_Autonomy 64. Exploring the role of agentic AI in fostering self-efficacy, autonomy support, and self-learning motivation in higher education - Frontiers, https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2026.1738774/full

65. Full article: The types of intrinsic motivation as predictors of academic achievement: the mediating role of deep learning strategy - Taylor & Francis, https://www.tandfonline.com/doi/full/10.1080/2331186X.2025.2482482 66. To See a Human: Intrinsic Motivation and Self-Determination | Psychology Today, https://www.psychologytoday.com/us/blog/positively-different/202602/to-see-a-human-intrinsic-motivation-and-self-determination